

Proyecto 1. Redes Neuronales Convolucionales

Angel Emanuel Rodriguez Roman

November 2025

1 Introduction

El Deep Learning es un area dentro de Machine Learning que ha tenido gran éxito en diferentes ámbitos. Siendo la tarea de clasificación de imágenes una de sus aplicaciones fundamentales. Esta tarea implica asignar una etiqueta de clase a una imagen de entrada, y es la base de sistemas más complejos como el reconocimiento facial, la conducción autónoma y el diagnóstico médico. Tradicionalmente, este desafío ha sido abordado mediante técnicas de Machine Learning basadas en extracción manual de características. Sin embargo, con el advenimiento del Deep Learning, las Redes Neuronales Convolucionales (CNNs), son un tipo de red neuronal artificial que se utiliza principalmente para el análisis de imágenes (LeCun et al., 1998). Esta arquitectura se ha consolidado como el estándar de oro debido a su capacidad intrínseca para aprender automáticamente jerarquías de características visuales directamente de los datos en bruto. El presente trabajo se centra en el dataset CIFAR-10, un punto de referencia ampliamente utilizado en la investigación de Deep Learning que consta de 60,000 imágenes a color de 32×32 píxeles, distribuidas en 10 clases. Utilizaremos este conjunto de datos para llevar a cabo un estudio comparativo, analizando el rendimiento y la eficiencia de diferentes arquitecturas neuronales. El objetivo principal de este estudio es realizar un análisis comparativo riguroso del desempeño de las Redes Neuronales Convolucionales (CNNs) frente a los Perceptrones Multicapa (MLPs) simples en el contexto de la clasificación de imágenes.

A pesar del éxito de las CNNs en tareas de visión por computadora, resulta interesante cuantificar y mostrar empíricamente la deficiencia de las arquitecturas tradicionales (MLPs) para el manejo de datos de imágenes. Por lo que podríamos hacernos la pregunta fundamental ¿Cómo una pequeña modificación arquitectónica (la convolución) puede generar una ganancia de rendimiento tan drástica?

Por otra parte, a partir de los resultados empíricos, se pretende ilustrar el costo computacional y la eficiencia de las CNNs y MLPs. Y poder observar como los MLPs, a pesar de ser considerados como aproximadores universales, no destacan en tareas de imágenes, ya que tratan cada píxel de la imagen como una característica independiente, perdiendo la información de localidad y estructura

espacial.

La motivación es validar experimentalmente cómo la invarianza a la traslación (una propiedad intrínseca de la convolución) reduce drásticamente el número de parámetros necesarios en la red, lo que lleva a un entrenamiento más rápido y a una menor necesidad de datos en comparación con el MLP, que debe aprender la misma característica en cada ubicación diferente de la imagen. En ese sentido, se establecerá el MLP como un punto de referencia (baseline) en la clasificación de imágenes. Esto ayuda a medir el verdadero valor añadido por las capas convolucionales, sirviendo como una prueba de concepto fundamental en cualquier proyecto de visión por computadora.

2 Metodología y arquitecturas utilizadas

El dataset CIFAR-10 cuenta con 60,000 imágenes a color de 32×32 píxeles, distribuidas en 10 clases (plane, car, bird, cat, deer, dog, frog, horse, ship, truck).

El dataset está particionado en dos conjuntos, de la siguiente manera:

- 50,000 imágenes para el conjunto de entrenamiento
- 10,000 imágenes para el conjunto de prueba

Experimento 1

Para el experimento 1. Se tomó únicamente el conjunto de entrenamiento, es decir, 50,000 imágenes, de las cuales se formó una partición de tres conjuntos (entrenamiento, validación y prueba), con los porcentajes 80,10,10. Para fines de reproducibilidad, se eligió como semilla aleatoria el número **42** para la partición del conjunto. Luego entonces, los conjuntos de datos quedan de la siguiente manera:

- 40,000 imágenes para el conjunto de entrenamiento
- 5,000 imágenes para el conjunto de validación
- 5,000 imágenes para el conjunto de prueba

Como parte del setup del experimento:

- Optimizador Adam con $lr = 1e^{-3}$, y $weight_decay = 0.0005$
- Batch size de 64
- Data augmentation (Random flips y crops)

Experimento 2

Para el experimento 2. Se tomó el dataset completo, es decir, 60,000 imágenes, de las cuales se formó una partición de tres conjuntos (entrenamiento, validación y prueba), con los porcentajes 80,10,10. Para fines de reproducibilidad, se eligió como semilla aleatoria el número **42** para la partición del conjunto. Luego entonces, los conjuntos de datos quedan de la siguiente manera:

- 48,000 imágenes para el conjunto de entrenamiento
- 6,000 imágenes para el conjunto de validación
- 6,000 imágenes para el conjunto de prueba

Como parte del setup del experimento:

- Optimizador Adam con $lr = 1e^{-3}$, y $weight_decay = 0.0005$
- Batch size de 64
- Data augmentation (Random flips y crops)

Las arquitecturas utilizadas para **ambos experimentos** fueron las siguientes:

MLP

Total de Parámetros Entrenables: 1,738,890

- Capa de entrada: 3072 Neuronas
- Capa oculta 1: $512 \rightarrow 256$ Neuronas
- Capa oculta 2: $256 \rightarrow 128$ Neuronas
- Capa de salida: $128 \rightarrow 10$ Neuronas
- Activación en cada salida: ReLU
- Regurización: Dropout = 0.5

CNN 1

Total de Parámetros Entrenables: 160,906

Table 1: Descripción de la Arquitectura de la CNN (CNN1)

Bloque / Etapa	Tipo de Capa	Parámetros	Salida
Entrada	Input	3 Canales	(3, 32, 32)
Bloque 1	CONV2D	$3 \rightarrow 32$ filtros, $k = 3$, $p = 1$	(32, 32, 32)
	BatchNorm2D	32 canales	(32, 32, 32)
	ReLU	-	(32, 32, 32)
	Dropout2D	$p = 0.1$	(32, 32, 32)
	MaxPool2D	$k = 2$, $s = 2$	(32, 16, 16)
Bloque 2	CONV2D	$32 \rightarrow 64$ filtros, $k = 3$, $p = 0$	(64, 14, 14)
	BatchNorm2D	64 canales	(64, 14, 14)
	ReLU	-	(64, 14, 14)
	Dropout2D	$p = 0.1$	(64, 14, 14)
	MaxPool2D	$k = 2$, $s = 2$	(64, 7, 7)
Bloque 3	CONV2D	$64 \rightarrow 128$ filtros, $k = 3$, $p = 0$	(128, 5, 5)
	BatchNorm2D	128 canales	(128, 5, 5)
	ReLU	-	(128, 5, 5)
	Dropout2D	$p = 0.1$	(128, 5, 5)
	MaxPool2D	$k = 2$, $s = 2$	(128, 2, 2)
Capas Densas	Flatten	-	512 neuronas
	Linear ($l1$)	$512 \rightarrow 128$ neuronas	128 neuronas
	BatchNorm1D	128 neuronas	128 neuronas
	ReLU	-	128 neuronas
	Dropout	$p = 0.5$	128 neuronas
	Linear ($l2$)	$128 \rightarrow 10$ neuronas	10 neuronas

CNN 2

Total de Parámetros Entrenables: 1,343,146

Table 2: Descripción de la Arquitectura de la CNN (CNN2)

Bloque / Etapa	Tipo de Capa	Parámetros Clave	Salida
Entrada	Input	3 Canales (RGB)	(3, 32, 32)
Bloque 1	CONV2D _{1,1}	3 → 32 filtros, $k = 3$, $p = 1$	(32, 32, 32)
	BatchNorm2D	32 canales	(32, 32, 32)
	CONV2D _{1,2}	32 → 32 filtros, $k = 3$, $p = 1$	(32, 32, 32)
	BatchNorm2D	32 canales	(32, 32, 32)
	MaxPool2D	$k = 2$, $s = 2$	(32, 16, 16)
	Dropout2D	$p = 0.1$	(32, 16, 16)
	ReLU	Despues de cada BN	-
Bloque 2	CONV2D _{2,1}	32 → 64 filtros, $k = 3$, $p = 1$	(64, 16, 16)
	BatchNorm2D	64 canales	(64, 16, 16)
	CONV2D _{2,2}	64 → 64 filtros, $k = 3$, $p = 1$	(64, 16, 16)
	BatchNorm2D	64 canales	(64, 16, 16)
	MaxPool2D	$k = 2$, $s = 2$	(64, 8, 8)
	Dropout2D	$p = 0.2$	(64, 8, 8)
	ReLU	Despues de cada BN	-
Bloque 3	CONV2D _{3,1}	64 → 128 filtros, $k = 3$, $p = 1$	(128, 8, 8)
	BatchNorm2D	128 canales	(128, 8, 8)
	CONV2D _{3,2}	128 → 128 filtros, $k = 3$, $p = 1$	(128, 8, 8)
	BatchNorm2D	128 canales	(128, 8, 8)
	MaxPool2D	$k = 2$, $s = 2$	(128, 4, 4)
	Dropout2D	$p = 0.25$	(128, 4, 4)
	ReLU	Despues de cada BN	-
Capas Densas	Flatten	-	$128 \times 4 \times 4 = 2048$ neuronas
	Linear ($l1$)	2048 → 512 neuronas	512 neuronas
	BatchNorm1D	512 neuronas	512 neuronas
	Dropout	$p = 0.6$	512 neuronas
	Linear ($l2$)	512 → 10 neuronas	10 neuronas

3 Resultados experimento 1

MLP

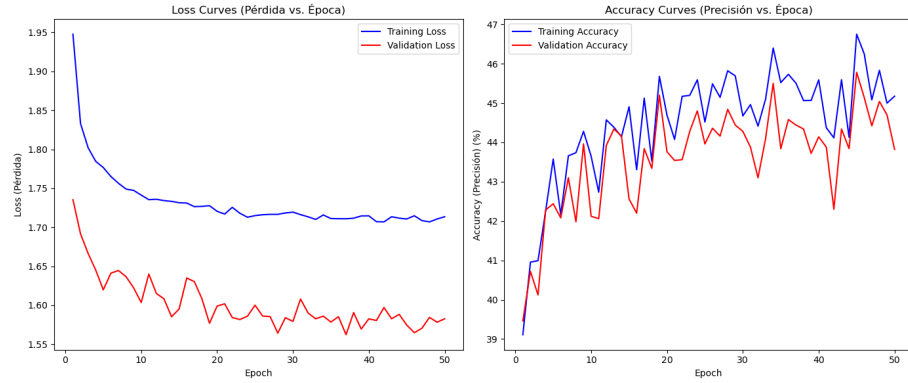


Figure 1: Curvas de aprendizaje para el Experimento 1, Arquitectura MLP. Se observa que la pérdida en el conjunto de validación es menor que en el de entrenamiento, lo cual podría deberse a que el modelo no logra alcanzar valores satisfactorios de accuracy. Esto sugiere que los pesos no se han ajustado de manera efectiva, manteniéndose en gran medida aleatorios. Las curvas de accuracy de entrenamiento y validación presentan un comportamiento similar, aunque ambas se mantienen por debajo del 50%.

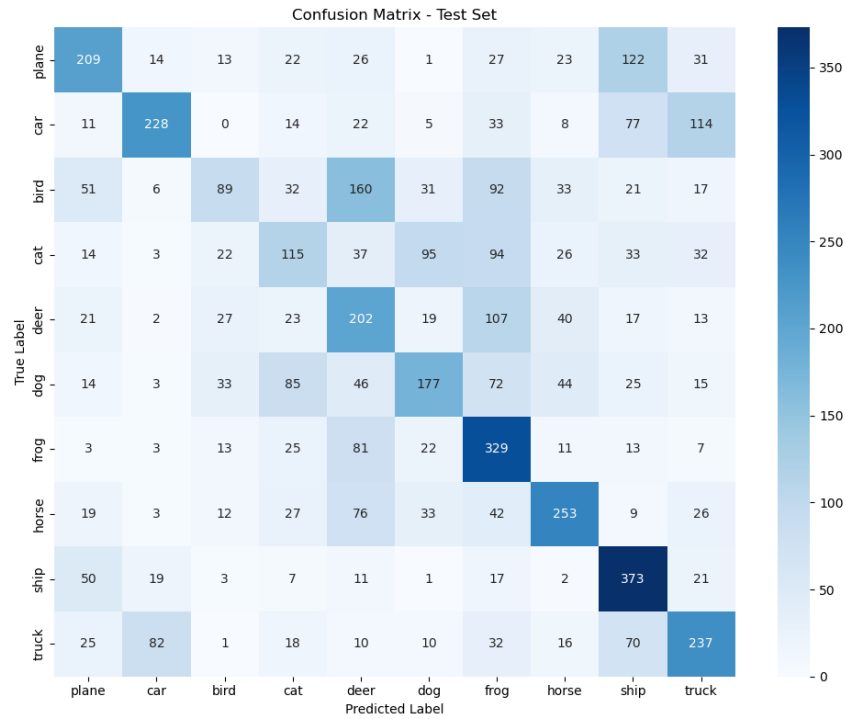


Figure 2: Matriz de confusión de la clasificación del MLP en el Experimento 1. Se observa un número considerable de errores en la clasificación. Por ejemplo, la clase bird presenta una alta confusión con la clase deer, lo que indica que el modelo tiene dificultades para distinguir entre ambas. Sin embargo, se obtienen resultados destacables en ciertas categorías, como frog y ship, donde el modelo logra una clasificación más precisa.

Table 3: Reporte de Clasificación por Clase (MLP Simple) - Experimento 1

Clase	Precision	Recall	F1-Score	Soporte
plane	0.50	0.43	0.46	488
car	0.63	0.45	0.52	512
bird	0.42	0.17	0.24	532
cat	0.31	0.24	0.27	471
deer	0.30	0.43	0.35	471
dog	0.45	0.34	0.39	514
frog	0.39	0.65	0.49	507
horse	0.55	0.51	0.53	500
ship	0.49	0.74	0.59	504
truck	0.46	0.47	0.47	501
Accuracy	0.44			5000

Table 4: Análisis Comparativo de Rendimiento: GPU vs. CPU (MLP) - Experimento 1

Evaluación	GPU	CPU	Factor de Aceleración (GPU/CPU)
FASE 1: Entrenamiento			
Tiempo Total de Entrenamiento (s)	1196.07	1622.10	1.36 × (GPU más rápida)
Tiempo Promedio por Época (s)	11.68	19.90	1.70 × (GPU más rápida)
FASE 2: Inferencia (Pruebas)			
Tiempo Total de Inferencia (s)	1.58	1.35	0.85 × (CPU más rápida)
Predicciones por Segundo (img/s)	3167.65	3709.19	0.85 × (CPU más rápida)

CNN 1

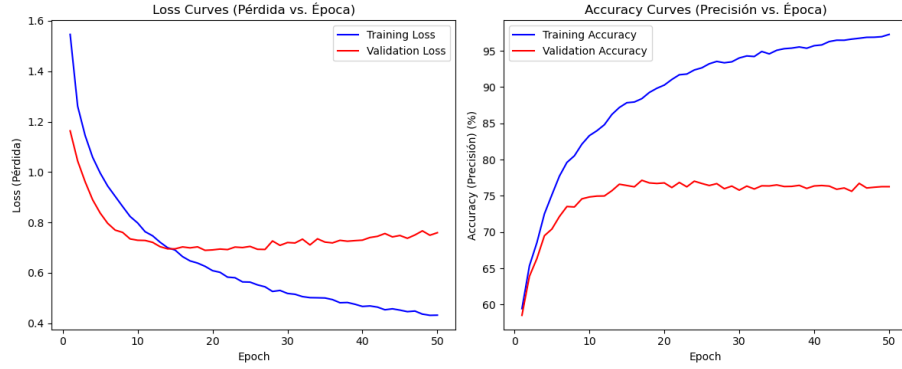


Figure 3: Curvas de aprendizaje para el Experimento 1, Arquitectura CNN1. Se observa una mejora significativa respecto al MLP, sin embargo, existe una disparidad significativa entre los resultados para los conjuntos de entrenamiento y validación. La pérdida en validación exhibe un valor inferior al de entrenamiento al inicio. Sin embargo, a partir de la época 15 aproximadamente, la pérdida de validación se mantiene en incremento de manera suave. En términos de precisión, el modelo alcanza un rendimiento superior al 95 % en entrenamiento, pero solo ≈ 78 % en validación, lo que confirma un claro sobreajuste (overfitting) desde las primeras épocas.

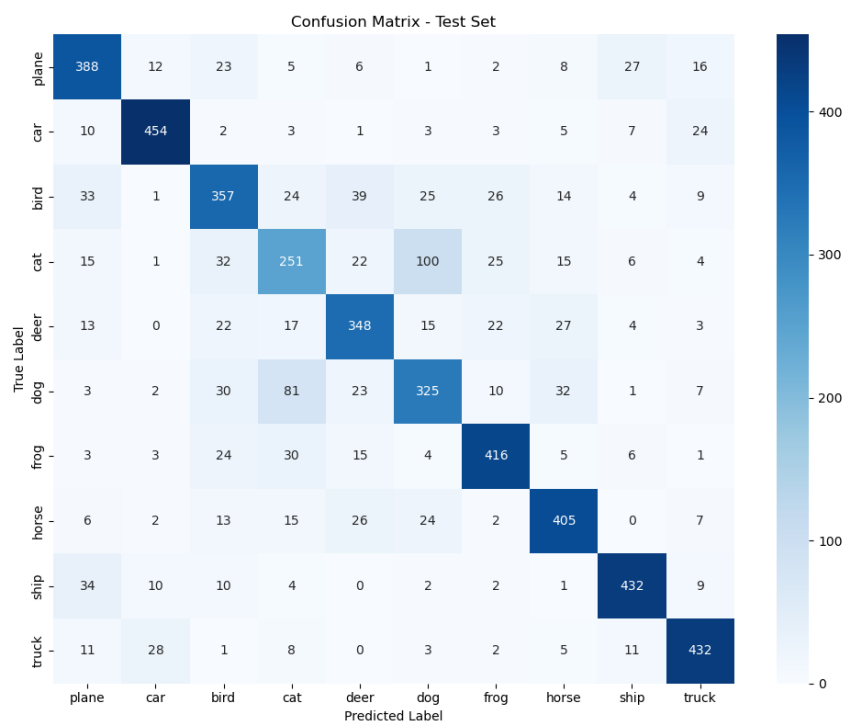


Figure 4: Matriz de confusión de la CNN1 para el experimento 1
Se observa una mejora en la clasificación para todas las clases con respecto a la matriz generada por el MLP. Para la gran mayoría de las clases, se obtienen buenos resultados (la diagonal de la matriz).

Table 5: Reporte de Clasificación por Clase (CNN1) - Experimento 1

Clase	Precision	Recall	F1-Score	Soporte
plane	0.75	0.80	0.77	488
car	0.88	0.89	0.89	512
bird	0.69	0.67	0.68	532
cat	0.57	0.53	0.55	471
deer	0.72	0.74	0.73	471
dog	0.65	0.63	0.64	514
frog	0.82	0.82	0.82	507
horse	0.78	0.81	0.80	500
ship	0.87	0.86	0.86	504
truck	0.84	0.86	0.85	501
Accuracy	0.76			5000

Profiling CPU vs GPU

Table 6: Comparativa de Rendimiento: GPU vs. CPU (CNN 1) - Experimento

Evaluación	GPU	CPU	Factor de Aceleración ($\times$)
FASE 1: Entrenamiento			
Tiempo Total de Entrenamiento (s)	1911.03	3382.67	1.77 $\times$
Tiempo Promedio por Época (s)	27.44	43.19	1.57 $\times$
FASE 2: Inferencia (Pruebas)			
Tiempo Total de Inferencia (s)	1.11	2.70	2.43 $\times$
Predicciones por Segundo (img/s)	4511.90	1849.11	2.44 $\times$

CNN 2

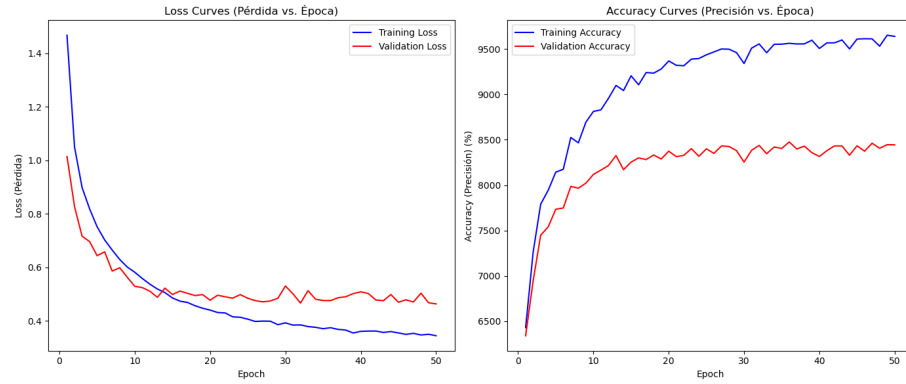


Figure 5: Curvas de aprendizaje para el Experimento 1, Arquitectura CNN2. Tanto la pérdida del conjunto de entrenamiento, como validación, muestran un comportamiento similar. Lo que nos dice que el modelo está aprendiendo de manera correcta. En términos de precisión, el modelo alcanza un rendimiento superior al 95 % en entrenamiento, y $\approx 85\%$ en validación. Aunque la diferencia de ≈ 10 puntos porcentuales aún indica un grado de sobreajuste, la mejora significativa de 78 % a 85 % con respecto a la arquitectura anterior (CNN1) demuestra una mayor robustez y capacidad de generalización de la CNN2, justificando su diseño arquitectónico.

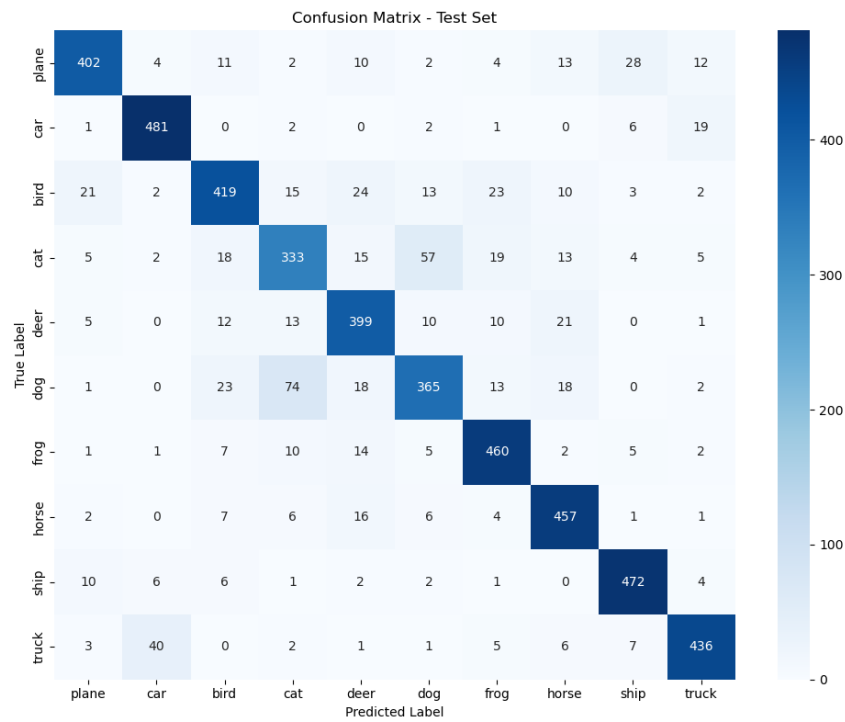


Figure 6: Matriz de confusión de la CNN2 para el experimento 1
 Como era de esperar, se obtienen resultados superiores a la CNN1, para todas clases, se presenta un mejora en accuracy, a pesar de que en algunas casos, hay un aumento de clasificaciones incorrectas, como el caso de truck, que es clasificado incorrectamente como car 28 veces con la CNN1. Y 40 con la CNN2. Sin embargo, el valor de accuracy, sigue siendo ligeramente mejor en la CNN2 (Ver valor en la diagonal)

Table 7: Reporte de Clasificación por Clase (CNN2) - Experimento 1

Clase	Precision	Recall	F1-Score	Soporte
plane	0.89	0.82	0.86	488
car	0.90	0.94	0.92	512
bird	0.83	0.79	0.81	532
cat	0.73	0.71	0.72	471
deer	0.80	0.85	0.82	471
dog	0.79	0.71	0.75	514
frog	0.85	0.91	0.88	507
horse	0.85	0.91	0.88	500
ship	0.90	0.94	0.92	504
truck	0.90	0.87	0.89	501
Accuracy	0.84			5000

Profiling CPU vs GPU

Table 8: Análisis Comparativo de Rendimiento: GPU vs. CPU (CNN 2)

Evaluación	GPU	CPU	Factor de Aceleración (×)
FASE 1: Entrenamiento			
Tiempo Total de Entrenamiento (s)	1483.76	7068.29	4.76×
Tiempo Promedio por Época (s)	15.16	99.03	6.53×
FASE 2: Inferencia (Pruebas)			
Tiempo Total de Inferencia (s)	1.55	4.43	2.86×
Predicciones por Segundo (img/s)	3217.01	1127.40	2.85×

4 Resultados experimento 2

MLP

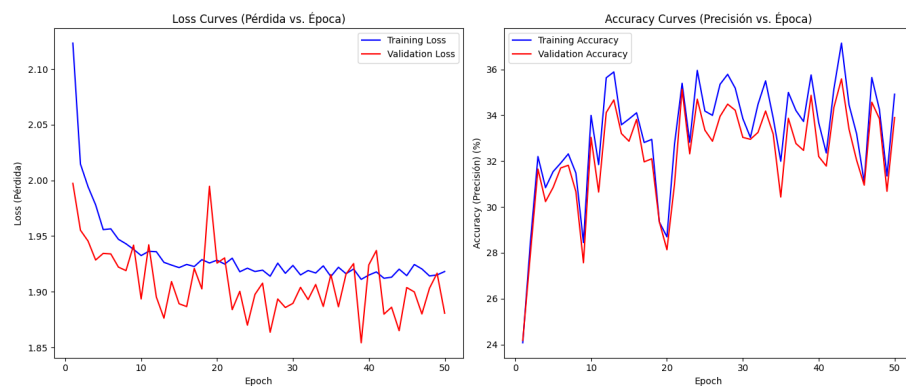


Figure 7: Curvas de aprendizaje para el Experimento 2, Arquitectura MLP. Se observan resultados similares al experimento 1. Sin embargo, hay una caída en accuracy y un aumento en la pérdida

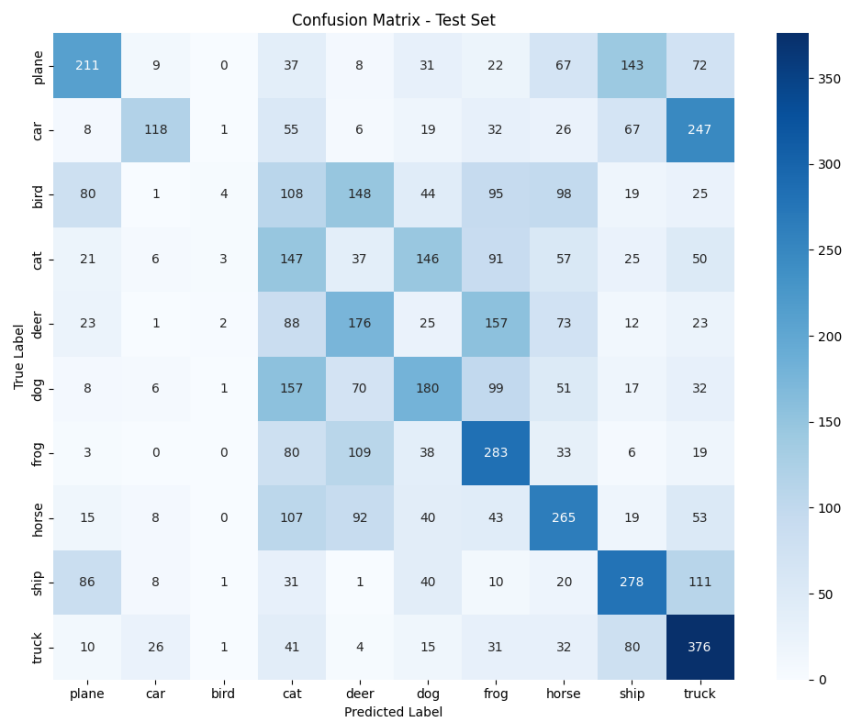


Figure 8: Matriz de confusión para el experimento 2, Arquitectura MLP

Table 9: Reporte de Clasificación por Clase (MLP), experimento 2

Clase	Precision	Recall	F1-Score	Soporte
plane	0.45	0.35	0.40	600
car	0.64	0.20	0.31	579
bird	0.31	0.01	0.01	622
cat	0.17	0.25	0.21	583
deer	0.27	0.30	0.29	580
dog	0.31	0.29	0.30	621
frog	0.33	0.50	0.39	571
horse	0.37	0.41	0.39	642
ship	0.42	0.47	0.44	586
truck	0.37	0.61	0.46	616
Accuracy	0.34			6000

CNN 1

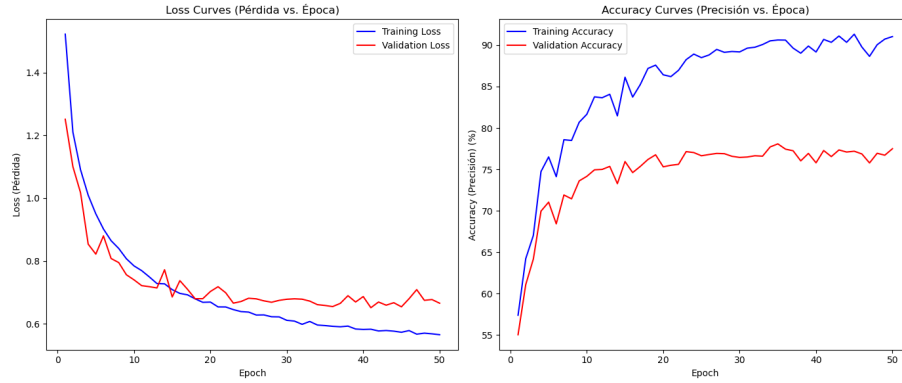


Figure 9: Curvas de aprendizaje para el Experimento 2, Arquitectura CNN1
 n A diferencia del experimento 1, se observa un mejor ajuste de los pesos, ya que la pérdida en el conjunto de validación, disminuye bastante y se acerca mucho a la entrenamiento. Se observa una mejora en accuracy y una mejor generalización lo cual se traduce también en una disminución del sobreajuste.

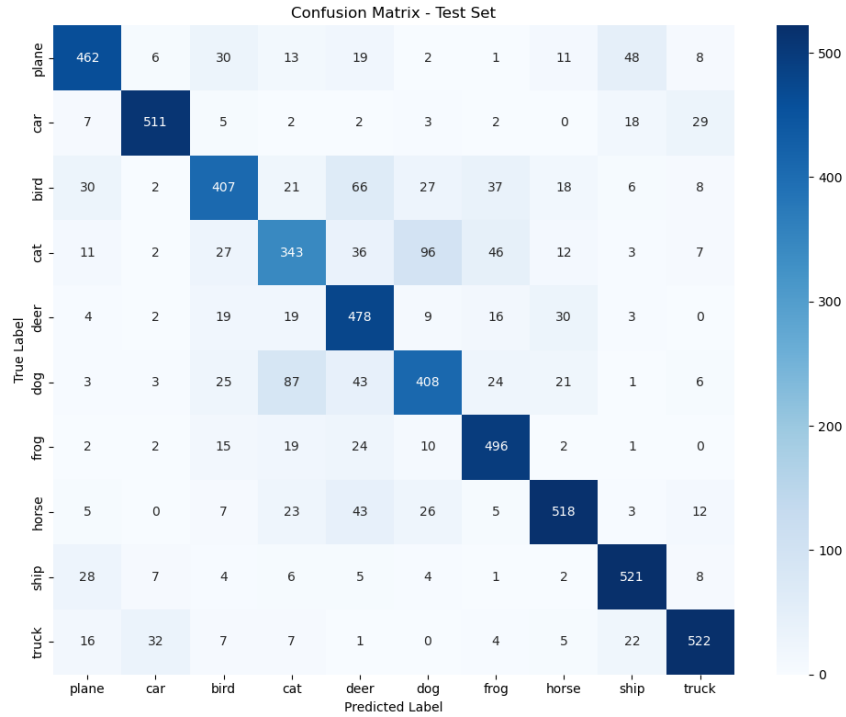


Figure 10: Matriz de confusi3n del Experimento 2, Arquitectura CNN1
n Se observa una mejora en accuracy para la mayor3a de las clases.

Table 10: Reporte de Clasificaci3n por Clase (CNN 1)

Clase	Precision	Recall	F1-Score	Soporte
plane	0.81	0.77	0.79	600
car	0.90	0.88	0.89	579
bird	0.75	0.65	0.70	622
cat	0.64	0.59	0.61	583
deer	0.67	0.82	0.74	580
dog	0.70	0.66	0.68	621
frog	0.78	0.87	0.82	571
horse	0.84	0.81	0.82	642
ship	0.83	0.89	0.86	586
truck	0.87	0.85	0.86	616
Accuracy	0.78			6000

Table 11: Análisis Comparativo de Rendimiento: GPU vs. CPU (CNN 1) - Experimento 2

Evaluación	GPU	CPU	Factor de Aceleración ($\times$)
FASE 1: Entrenamiento			
Tiempo Total de Entrenamiento (s)	1116.24	3350.97	3.00$\times$
Tiempo Promedio por Época (s)	11.41	44.24	3.88$\times$
FASE 2: Inferencia (Pruebas)			
Tiempo Total de Inferencia (s)	1.41	2.44	1.73$\times$
Predicciones por Segundo (img/s)	4246.45	2454.98	1.73$\times$

CNN 2

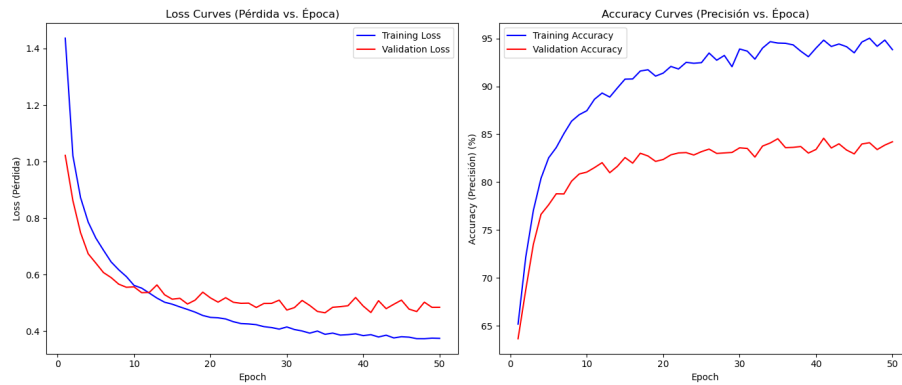


Figure 11: Curvas de aprendizaje para el Experimento 2, Arquitectura CNN2. Si bien, las curvas y los valores alcanzados en las diferentes épocas son muy parecidos, se puede observar una mejora tanto en la pérdida como en accuracy.

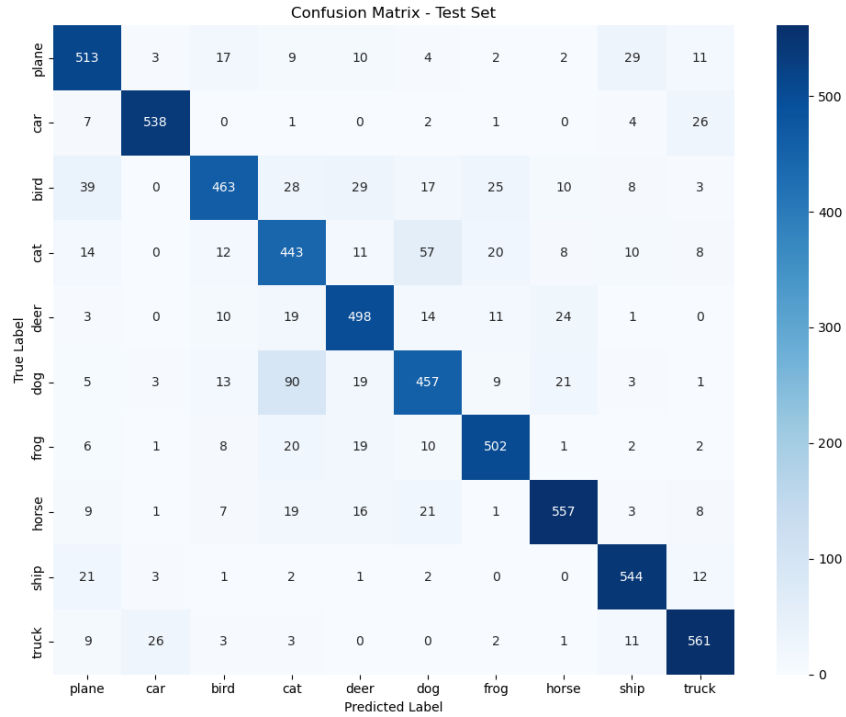


Figure 12: Matriz de confusión del Experimento 2, Arquitectura CNN2
Con este resultado podemos confirmar, que el desempeño de la CNN2 es superior al de la CNN1. Ya se que se logra un mejor accuracy en todas las clases.

Table 12: Reporte de Clasificación por Clase (CNN 2), Experimento 2

Clase	Precision	Recall	F1-Score	Soporte
plane	0.82	0.85	0.84	600
car	0.94	0.93	0.93	579
bird	0.87	0.74	0.80	622
cat	0.70	0.76	0.73	583
deer	0.83	0.86	0.84	580
dog	0.78	0.74	0.76	621
frog	0.88	0.88	0.88	571
horse	0.89	0.87	0.88	642
ship	0.88	0.93	0.91	586
truck	0.89	0.91	0.90	616
Accuracy	0.85			6000

Table 13: Análisis Comparativo de Rendimiento: GPU vs. CPU (CNN 2) - Experimento 2

Evaluación	GPU	CPU	Factor de Aceleración ($\times$)
FASE 1: Entrenamiento			
Tiempo Total de Entrenamiento (s)	1174.55	8052.76	6.86 $\times$
Tiempo Promedio por Época (s)	12.40	115.59	9.32 $\times$
FASE 2: Inferencia (Pruebas)			
Tiempo Total de Inferencia (s)	2.36	4.75	2.01 $\times$
Predicciones por Segundo (img/s)	2547.21	1264.23	2.01 $\times$

5 Conclusions

En el caso del MLP, los resultados obtenidos fueron en general bajos. Las curvas de entrenamiento muestran que, incluso sobre los propios datos de entrenamiento, el modelo presenta dificultades para aprender patrones relevantes, alcanzando valores de accuracy menores al 50 %. En términos prácticos, su rendimiento no supera el de una predicción aleatoria, a pesar de que el MLP es teóricamente un aproximador universal. Una posible explicación es que los datos de entrada, al ser imágenes aplanadas en vectores, pierden su información espacial y estructural, lo cual limita severamente la capacidad del modelo para generalizar. Podría pensarse que un mayor número de ejemplos de entrenamiento ayudaría, sin embargo, los resultados del Experimento 2 muestran lo contrario: el desempeño del modelo empeora incluso con más datos. Esto sugiere que el problema no radica únicamente en la cantidad de datos, sino en la forma en que el modelo los procesa. Sería interesante realizar más experimentos que analicen cómo influye el tamaño y la representación del conjunto de datos en el rendimiento del MLP.

Por otro lado, para la CNN1, la arquitectura propuesta presenta una cantidad relativamente baja de parámetros entrenables en comparación con el MLP utilizado. Aun así, logra valores de accuracy consistentemente superiores al 50 % en todos los casos. La principal diferencia radica en las capas convolucionales, las cuales aprovechan la operación de convolución para extraer características espaciales de forma jerárquica y son invariantes a traslaciones. Esto permite que un patrón aprendido en una región de la imagen sea útil para reconocerlo en otra.

Las curvas de aprendizaje muestran un desempeño significativamente mejor que en el MLP: la CNN alcanza valores superiores al 90 % de accuracy en el conjunto de entrenamiento. No obstante, se observa rápidamente el fenómeno de sobreajuste (overfitting), a pesar de aplicar técnicas de regularización y aumento de datos. Este comportamiento sugiere que, dada la complejidad de la arquitectura, podría ser necesario incrementar el tamaño del conjunto de entrenamiento para mitigar el sobreajuste. De hecho, los resultados del Experimento 2 apoyan esta hipótesis, mostrando mejoras en el rendimiento cuando se dispone de más

datos.

En resumen, los resultados confirman que las CNN son mucho más adecuadas para el procesamiento de imágenes que los MLP, debido a su capacidad de preservar y explotar la información espacial. Sin embargo, también evidencian la importancia del tamaño del conjunto de datos y de una adecuada regularización para evitar el sobreajuste.